CÓDIGOCLARO · MATERIAL DE ESTUDIO

SQL, MySQL y PL/SQL

Cuaderno DAM/DAW · MySQL 8.4 con InnoDB y Oracle AI Database 26ai · dialectos separados.

Modelo relacional y claves

Objetivo: Traducir requisitos de un módulo DAM a tablas con identificadores y relaciones.

La clave primaria identifica cada fila y la clave foránea mantiene una referencia válida. Una clave compuesta impide duplicar una relación concreta. Modela cardinalidades reales para evitar copiar el nombre del módulo en cada matrícula. Comprueba restricciones con filas válidas e inválidas, no solo con el caso normal.

Código · MySQL 8.4

```
CREATE TABLE alumno (
  id INT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL
);
CREATE TABLE matricula (
  alumno_id INT NOT NULL,
  modulo VARCHAR(60) NOT NULL,
  PRIMARY KEY (alumno_id, modulo),
  FOREIGN KEY (alumno_id) REFERENCES alumno(id)
);
```

Práctica: ¿Qué evita repetir una matrícula del mismo alumno en el mismo módulo?

Respuesta razonada: La clave primaria compuesta alumno_id y modulo hace única la pareja. La clave foránea exige que el alumno exista; NOT NULL impide una relación incompleta.

Normalización y dependencias

Objetivo: Reducir duplicidad al guardar datos de alumnos, módulos y profesorado.

Si el nombre del módulo aparece en muchas filas, cambiarlo obliga a modificar varias copias y puede dejar contradicciones. Separa atributos que dependen de claves distintas y enlázalos mediante identificadores. Normalizar no significa dividir por rutina: explica qué dependencia funcional evita cada tabla y verifica que una consulta pueda reconstruir el dato.

Código · MySQL 8.4

```
CREATE TABLE modulo (
  id INT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL
);
CREATE TABLE matricula (
  alumno_id INT NOT NULL,
  modulo_id INT NOT NULL,
  PRIMARY KEY (alumno_id, modulo_id)
);
```

Práctica: Si cambia el nombre de un módulo, ¿cuántas filas se actualizan con este diseño?

Respuesta razonada: Se actualiza una fila de modulo; las matrículas conservan el identificador. Se elimina la anomalía de actualización sin duplicar el nombre en cada inscripción.

SELECT, GROUP BY y HAVING

Objetivo: Calcular medias por módulo separando filtros de fila y filtros de grupo.

WHERE se aplica antes de agrupar; GROUP BY define grupos y HAVING filtra esos grupos ya calculados. Incluye en la agrupación las columnas no agregadas y ordena por el criterio del informe. Escribe primero la población de filas que quieres analizar y luego el agregado para no mezclar convocatorias.

Código · MySQL 8.4

```
SELECT modulo_id, AVG(nota) AS media
FROM resultado
WHERE convocatoria = 'ordinaria'
GROUP BY modulo_id
HAVING COUNT(*) >= 5
ORDER BY media DESC;
```

Práctica: ¿Qué ocurre con un módulo que tiene solo cuatro notas ordinarias?

Respuesta razonada: No aparece: WHERE conserva las notas ordinarias y HAVING elimina grupos con menos de cinco. AVG se calcula por módulo, no sobre toda la tabla.

JOIN y cardinalidad

Objetivo: Conservar alumnos aunque todavía no tengan matrículas.

INNER JOIN conserva pares coincidentes; LEFT JOIN conserva cada fila izquierda y rellena con NULL las columnas derechas sin pareja. Una relación uno a muchos repite la fila de la tabla izquierda. Revisa la cardinalidad y el efecto de los filtros posteriores, porque un WHERE sobre la tabla derecha puede eliminar los nulos.

Código · MySQL 8.4

```
SELECT a.id, a.nombre, m.modulo_id
FROM alumno AS a
LEFT JOIN matricula AS m
  ON m.alumno_id = a.id
ORDER BY a.id, m.modulo_id;
```

Práctica: ¿Qué ve una consulta para un alumno sin matrículas?

Respuesta razonada: La fila de alumno permanece y modulo_id vale NULL. Si ese campo se filtra después con WHERE, la fila puede desaparecer; el filtro debe situarse según el resultado que exige el enunciado.

Subconsultas y CTE

Objetivo: Nombrar un conjunto intermedio al construir un informe de módulos con promedio suficiente.

Una subconsulta o un CTE permite separar pasos de una consulta. WITH define un resultado temporal dentro de una sola sentencia, no una tabla persistente. Selecciona solo columnas necesarias y mantén la misma granularidad entre el cálculo y la salida. Un CTE mejora lectura, pero no garantiza por sí solo un plan más rápido.

Código · MySQL 8.4

```
WITH medias AS (
  SELECT modulo_id, AVG(nota) AS media
  FROM resultado GROUP BY modulo_id
)
SELECT modulo_id, media FROM medias
WHERE media >= 7.0;
```

Práctica: ¿Por qué alumno_id no forma parte del GROUP BY del informe?

Respuesta razonada: El informe tiene una fila por módulo. Incluir alumno_id cambiaría la granularidad a una fila por alumno y produciría una media distinta.

DML y parámetros seguros

Objetivo: Cambiar una matrícula por identificador evitando construir SQL con texto de usuario.

INSERT crea filas, UPDATE modifica y DELETE elimina las seleccionadas. Antes de una operación masiva inspecciona el filtro con SELECT; usa parámetros del controlador y una transacción cuando haya varios cambios relacionados. Parámetros protegen la sintaxis frente a inyección, pero no validan la regla de negocio ni corrigen un WHERE demasiado amplio.

Código · JDBC + MySQL 8.4

```
UPDATE matricula
SET convocatoria = ?
WHERE alumno_id = ? AND modulo_id = ?;
-- El driver enlaza los valores como parámetros.
```

Práctica: El identificador del módulo llega vacío. ¿Debe ejecutarse la actualización?

Respuesta razonada: No. Valida la entrada y comprueba que el filtro apunta a la matrícula concreta. El enlace de parámetros evita inyección, pero la aplicación debe rechazar identificadores incompletos.

Transacciones e InnoDB

Objetivo: Mantener coherente el stock al confirmar una reserva.

Una transacción agrupa operaciones que deben confirmarse juntas o revertirse juntas. En InnoDB ACID y el nivel de aislamiento condicionan la concurrencia; el programa debe comprobar las filas afectadas. ROLLBACK descarta cambios aún no confirmados. No mezcles esta sintaxis MySQL con bloques PL/SQL de Oracle.

Código · MySQL 8.4 / InnoDB

```
START TRANSACTION;
UPDATE material SET unidades = unidades - 1
WHERE id = 12 AND unidades > 0;
-- Comprobar filas afectadas antes de confirmar.
COMMIT; -- o ROLLBACK si no se reservó
```

Práctica: La actualización afecta cero filas. ¿Qué hace el flujo con el resto de la reserva?

Respuesta razonada: Revierte la transacción e informa de que no quedan unidades. La condición evita descontar bajo cero, pero comprobar el número de filas afectadas decide si puede confirmarse la reserva completa.

Índices y EXPLAIN

Objetivo: Examinar un índice compuesto según las consultas reales del inventario.

Un índice acelera ciertas búsquedas pero añade coste al insertar y actualizar. En un índice compuesto importa el orden de columnas y los prefijos que puede usar el optimizador. EXPLAIN muestra el plan estimado; mide con datos representativos antes de añadir índices y comprueba si la consulta usa realmente la parte esperada.

Código · MySQL 8.4

```
CREATE INDEX ix_matricula_alumno_modulo
ON matricula (alumno_id, modulo_id);
EXPLAIN
SELECT modulo_id FROM matricula
WHERE alumno_id = 42;
```

Práctica: ¿El mismo índice sirve necesariamente para filtrar solo por modulo_id?

Respuesta razonada: No necesariamente: el prefijo empieza por alumno_id. Si ese patrón es frecuente, evalúa un índice con modulo_id primero y compara costes de consulta y escritura.

PL/SQL y SELECT INTO

Objetivo: Distinguir un bloque de Oracle de una consulta SQL MySQL.

PL/SQL añade variables, flujo y excepciones sobre SQL de Oracle. SELECT INTO asigna exactamente una fila: cero produce NO_DATA_FOUND y varias TOO_MANY_ROWS. Si se esperan varias filas, usa cursor o una operación agregada. La etiqueta del dialecto importa porque MySQL no ejecuta esta sintaxis de bloque.

Código · Oracle PL/SQL 26ai

```
DECLARE
  v_nombre employees.last_name%TYPE;
BEGIN
  SELECT last_name INTO v_nombre
  FROM employees WHERE employee_id = 100;
  DBMS_OUTPUT.PUT_LINE(v_nombre);
EXCEPTION
  WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No existe');
END;
/
```

Práctica: ¿Qué ocurre si el filtro no es único y encuentra varias filas?

Respuesta razonada: Se produce TOO_MANY_ROWS. SELECT INTO no convierte un conjunto plural en un escalar; cambia la consulta o procesa las filas mediante un cursor.

Rutinas, cursores y triggers

Objetivo: Elegir una herramienta PL/SQL según se necesite resultado, recorrido o reacción a DML.

Una función devuelve un valor; un procedimiento ejecuta una acción; un cursor permite procesar filas y un trigger reacciona a cambios DML. Prefiere SQL set-based cuando expresa la operación completa. Los triggers por fila disponen de :OLD y :NEW; evita confirmar la transacción desde el trigger.

Código · Oracle PL/SQL 26ai

```
CREATE OR REPLACE TRIGGER valida_stock
BEFORE INSERT OR UPDATE OF unidades ON material
FOR EACH ROW
BEGIN
  IF :NEW.unidades < 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Stock negativo');
  END IF;
END;
/
```

Práctica: ¿Cuándo se ejecuta el trigger y qué representa :NEW?

Respuesta razonada: Se ejecuta antes de insertar o cambiar unidades en cada fila. :NEW contiene el valor propuesto. La sintaxis es Oracle PL/SQL, no una declaración MySQL.

Fuentes oficiales

MySQL 8.4 Reference Manual · Oracle AI Database 26ai · SQL Language Reference · Oracle AI Database 26ai · PL/SQL Language Reference